Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 225 (2023) 4015–4023

Procedia Computer Science

www.elsevier.com/locate/procedia

27th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2023)

Adaptive dwell time for eye typing

Katarzyna Harezlak[a,*], Pawel Basek[a], Pawel Kasprowski[a]

[a]*Department of Applied Informatics, Silesian University of Technology Akademicka 16, 44-100 Gliwice, Poland*

Abstract

The presented studies investigated the possibility of eye movement usage for entering text using on-screen keyboards. The attention was focused on the keyboard layout and the time of typing a character. Three dwell times were used to select keyboard keys: 2-second, 1-second, and adaptive. The last of them required the development of an appropriate algorithm to adjust the character choice to the user's skills and to facilitate the use of the software. The paper analyzes whether the keyboard type and character selection time affect the text input speed and the number of errors made. During the experiment, in which seven people participated, the text consisting of approximately 70 characters was entered. This operation was repeated for each keyboard layout and dwell time. The studies have confirmed the reduced error number and the greater comfort for the adaptive selection solution.

© 2023 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 27th International Conference on Knowledge Based and Intelligent Information and Engineering Systems

Keywords: Eye typing; on-screen keyboard; adaptive dwell-time;

1. Introduction

Over the last decade, eye movement analysis has become the commonly used method to facilitate the assessment of people's intentions, interests, and skills. With the development of technology, more precise measurement of the eye position has become possible, which further allows for exploring such data according to the chosen purpose. Among systems utilizing eye tracking, *passive* and *active* applications can be distinguished. The first one only examines eye movements to reveal focused areas. The second group allows users to interact and control the program [22]. Such applications can still work with the keyboard and mouse [8]. Sometimes, applying eye-based steering eliminates the need to use other input devices enabling disabled people to utilize electronic devices and control them by gaze or through eye gestures [20, 2, 1].

The most natural selection method is to fix one's eyes on an on-screen object and gaze at it for a pre-defined dwell time. Three parameters for such an approach can be considered: dwell time, button size, and content placement [17, 6]. In this case, the main concern is dwell duration, which can cause the Midas touch problem – when a command

* Corresponding author.
E-mail address: katarzyna.harezlak@polsl.pl

1877-0509 © 2023 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 27th International Conference on Knowledge Based and Intelligent Information and Engineering Systems
10.1016/j.procs.2023.10.397

is activated by unconsciously initiated action invoked by involuntary blinks. Thus when designing dwell-controlled applications, the trade-off between speed and accuracy has to be considered. In [7, 10, 12, 19], the gazing duration was suggested to be between 450–1000ms. Another solution is to use gaze to select but not to activate an interface control. Voluntary blinks can be used to confirm the gaze-made choice [4]. The use of eye blink was also applied for constructing a more advanced solution in the form of a virtual on-screen keyboard that can be operated by closing the eyelids [9]. Moreover, the Midas touch problem may be solved by relatively simple eye gestures being eye movement sequences. Only a correct repeating of the defined configuration leads to the execution of the specified operation [1, 20].

Among the commonly used eye-based controlled applications are such, which utilize on-screen keyboards. They can take very different sizes, shapes, and available functions. Thus, there is still a necessity to constantly improve them and search for better, more convenient, and user-friendly solutions. In this research, the attention was focused on improving the efficiency of the usage of the previously developed keyboards [5] utilizing the gaze-based key selection method. As a proposed development, the adaptive dwell time was considered. A hypothesis was made that adaptive dwell time would facilitate text entry by shortening typing time and reducing the error number. It was assumed that this hypothesis would apply to all types of keyboards.

2. Adaptive dwell time

Dwell time is a parameter that determines how long the users must keep their eyes on a specific point to make a button selection. This parameter is essential in applications for entering text. In many research, a fixed, predetermined selection time is commonly used [7, 10, 12, 19, 14]. Although a constant value of this parameter can be a solution for a particular user, it can be inconvenient for a group of people using the same typing software. Beginners getting acquainted with an eye tracker and keyboard may have some problems with memorizing the keys' layout or efficient equipment usage. They will need a longer selection time to avoid undesirable choices. On the other hand, for more advanced users, a shorter time would be more convenient. Longer one may cause impatience and may lead to errors. The adaptive selection time approach is a solution that can alleviate the mentioned problems. It allows the application to adjust the value of this parameter to the current user's typing efficiency.

Some studies have been conducted so far in this field. Majaranta et al. [11] provided a solution where the users could manually adjust the dwell time for selecting a key to their preferred value at any time during the experiment. The research presented in [21] used a similar approach to assess the influence of changing the key selection time on typing performance.

The solution that dynamically changes dwell time for emulating mouse click events for each user and application button was described in [16]. It applies different reward values for genuine dwell-clicks and Midas touches to evaluate the selection time. On the other hand, Mott et al. [15] proposed cascading gaze typing, which dynamically adjusts the dwell time for keys based on the probability that a key will be selected next and its location on the keyboard. This approach makes unlikely keys more challenging and likely keys easier to choose by increasing and decreasing dwell times, respectively. A similar method was proposed in [16] where the adjustment of dwell times is evaluated employing the user's background regarding different parts of a particular application. Moreover, the same approach was applied in [18]. It also considers the probability of each letter selection utilizing its past choices. It automatically assigns shorter dwell times to more likely characters and longer ones to less likely letters.

Another approach is presented in [13]. The main idea of the described algorithm here is to monitor the program's current state and adjust the selection time based on the collected information about errors and input speed. The more mistakes the user makes, the longer the selection time will be, while in the case of fast and error-free typing, this time will decrease. If the number of errors made divided by the number of all typing actions is greater than the given threshold, it means that the user has some problems, and the time of gaze may be increased. On the other hand, if the average time between two consecutive commands in the action history is close to the current dwell time, it means that typing is efficient and time can be reduced. This approach was adopted and slightly modified in the presented studies.

3. Modified adaptive dwell time algorithm

Independently of the approach used, all the aforementioned adaptive solutions improved typing efficiency and user typing convenience. It was the motivation factor to extend the previously conducted studies [5] with an adaptive dwell time for the key selection.

When developing an appropriate algorithm, several issues were taken into account. The first regarded determining the time between two actions, and two approaches were considered. One solution could represent the period between any two typing operations, and the second one, the time between inserting a character. The next focused issue was the list of actions included in the history – if it should exclude deleting and undo operations or, as in the basic algorithm, should consist of all button choices.

The assumptions, which were made, slightly modified the previously described adaptive selection time approach [13]. The first change was to separate typing operations into those that enter a character into the text field, actions enabling selecting a character, and delete and undo functions. This change was necessary to create the action history based only on those entering characters into a text field. The typing duration was defined as the period between the state that allows the letter to be selected and the action of choosing it. Another assumption was to count the undo and delete actions only to the group of errors, so when committing them, the general action counter was not incremented. The algorithm implementation is presented in pseudocode 1. The *step* parameter represents the increment and decrement of the dwell time in ms, and $th1$, $th2$ are the thresholds conditioning decreasing and increasing the selection time. The first algorithm part checks if the ratio of all errors to all entered characters is greater than the parameter $th2$ and if the maximum value of the selection time has not been reached. If both conditions are fulfilled, the selection time is increased by the *step*. In the subsequent decision block, the algorithm based on *History* calculates the average selection time and checks if this value is close to the duration of the last character choice. If yes, and the minimum adopted parameter value allows it, the dwell time is decreased by the *step*.

4. Method

Three typing methods, represented by various keyboard implementations, were used in this research. They are similar in entering text procedure but differ in the key layout and realization of some actions. All keyboards utilize the dwell-based button selection for typing text and require two steps for entering the letter with eyes. During the first one, a group of letters with the one intended to be entered is chosen. In the second step, an appropriate character is pointed out. The selected letter is added to a textbox presenting the text typed so far. Such an approach save space on the screen and allows for making buttons with characters larger than in the classic keyboard layout. However, entering one character requires two actions to be performed. As mentioned, the keyboards vary in some features, such as button shapes and arrangements, and an area for displaying text, which is visible in Figures 1, 2, 3.

As seen, the first keyboard, in Figure 1, proposed by Huckauf and Urbina [19] (later referred to as Pie), groups letters into the pie items. At first, only one pie with letter groups is visible on the screen. Another one with one sector per character appears after a given group is pointed out by gaze. On the right side of the keyboard, a textbox is placed in which the written text is visible. The Back button is located in the pie center, and the Delete button for correcting errors is available at the bottom pie item.

The keyboard in Figure 2, called Tree (following the keyboard authors [3]), includes rectangle command buttons, which at first show letter groups and, after selecting the required one, display characters belonging to the group. The letters are organized alphabetically, and the Delete button switches to the Back in the letter-choosing state. The textbox displaying the entered text is in the middle of the keyboard.

The last presented approach, in Figure 3, is denoted by Side because elements displaying groups and single letters are placed on the left and right sides of the screen, respectively [5]. The buttons are triangle-shaped due to the optimization of the space. At the beginning, only controls on the left side are visible. After selecting a group, the right panel becomes accessible with the characters arranged in the same order as in the chosen group on the left. The Delete operation is available in one of the bottom items in the left panel. However, the Back button is unnecessary because, in the case of the wrong group selection, users do not have to change the current view but only direct their gaze to the proper triangle on the left.

The detailed description of each solution is presented in [5].

Algorithm 1 Adaptive selection time – algorithm pseudocode

▷ Initial parameter values

$minSelectionTime \leftarrow 1000$
$maxSelectionTime \leftarrow 5000$
$SelectionTime \leftarrow 2000$
$step \leftarrow 500$
$th1 \leftarrow 700$
$th2 \leftarrow 0.5$
...

▷ If the number of mistakes divided by the number of properly entered characters is greater than the threshold,
▷ it means that a user has a problem with typing
▷ – increase selectionTime by the step.

if $\frac{Mistakes}{History} > th2$ **then**
  if $selectionTime + step < maxSelectionTime$ **then**
    $selectionTime = selectionTime + step$
  end if
end if

▷ If the difference between the average selectionTime in the history and the current selectionTime is lower
▷ than a given threshold, it means that a user types fluently
▷ – decrease selectionTime by the step.

if $(averageSelectionTimes(History)) - selectionTime) \leq th1$ **then**
  if $selectionTime - step > minSelectionTime$ **then**
    $selectionTime = selectionTime - step$
  end if
end if
return selectionTime

5. Experiment and Results

Seven participants took part voluntarily in the experiment, including four women and three men. The average age was 30 years, and the maximal 53. No person wore glasses or contact lenses and was familiar with eye-tracking technology. The experiments started with a calibration process based on nine points. All tests were realized using a Lenovo Y-700 laptop with an Intel(R) Core(TM) i7-6700HQ CPU @ 2.60GHz and 8GB RAM. The application was displayed on an LG monitor at a resolution of 1920x1080. The eye tracker used in the study was The Eye Tribe, which operated at a 60Hz sampling rate.

Initially, each user was familiarized with the application and the task relying on entering text (approximately 70 characters) read by the operator. All participants used all keyboards three times – once in the one-second selection mode, once in the two-second one, and once using the adaptive solution. Several hypotheses were made regarding the last-mentioned approach. At first, it was assumed that when using this approach, the typing speed would decrease for the two-second gaze time but would not fall below one second according to the algorithm settings. Additionally, it was expected that the error number would decrease for the adaptive time mode. Finally, it was hypothesised that the aforementioned assumptions would regard all keyboard types.

The order in which the keyboards were used differed in all tests. It was to eliminate the results being influenced by getting experience always using the same keyboard in the first place. When conducting the experiment, it was ensured

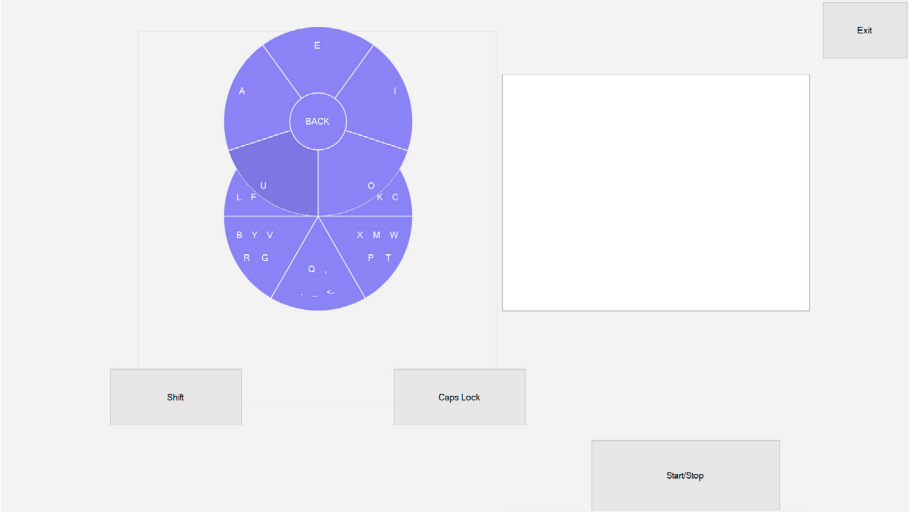


Fig. 1. Pie keyboards used in the research



Fig. 2. Tree keyboards used in the research

that the environment and conditions were identical for all participants. The average experiment time for each person was about 2 hours, including an introduction to the tests, task realization, and completion of the questionnaire.

The first analysed metric was the average typing speed, understood as the number of entered characters per minute. It was explored for each keyboard and each dwell time. The obtained results are presented in Table 1. As can be seen, the lowest typing speed, as expected, was achieved for the two-second key selection and the highest for the one-second one. It was related to the period of gazing at the button needed to accept the choice. With its results, the adaptive approach was placed between both static values, confirming that during the experiment, the participants were more acquainted with eye-typing and keyboard layout, thus improving their performance with time. Compared to the two-second dwell time, the most considerable improvement was achieved for the Pie keyboard.

The second explored metric was the error number. The mistakes affect the typing duration because they must be corrected, which increases the time needed to enter the required character. The results of the analysis conducted in terms of this aspect are presented in Table 2.

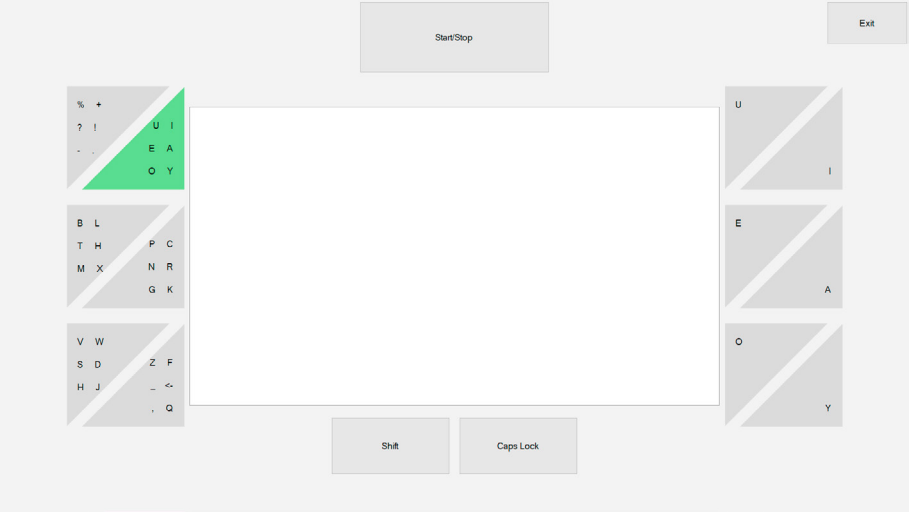


Fig. 3. Side keyboards used in the research

Table 1. The typing speed measured in the number of characters per minute.

Keyboard	Pie keyboard			Tree keyboard			Side keyboard		
Selection time [s]	2	1	Adaptive Time	2	1	Adaptive Time	2	1	Adaptive Time
Letters per minute	7.99	11.49	10.01	8.41	11.5	9.92	8.19	12.75	9.09

Table 2. The number of mistakes for each keyboard and selection time.

	Two-second selection time	One-second selection time	Adaptive selection time
Pie keyboard	165	272	111
Tree keyboard	29	86	19
Side keyboard	86	175	43
Total for selection time	**280**	**533**	**173**

As can be observed for the Pie keyboard, the number of mistakes committed with its usage is the largest. It may result from too small buttons placed too close to each other to be effectively chosen by gaze. In the case of the Tree keyboard, mistakes were the least frequent. It probably stems from the intuitiveness of the alphabetical arrangement of the letters and the large spacing between the buttons themselves. The Side approach is placed in the middle. However, it could be expected that outcomes would be better when using the alphabetical order. The obtained results' graphical representation for both metrics is presented in Figure 4.

Studying the results concerning the selection time, it becomes visible that the shortest required gaze duration caused the most errors. During the experiments, many participants complained about the two-second gazing as a too-long selection time, while the one-second one was too sensitive for some of them.

As presented in Table 2, a good solution to this problem was the application of the adaptive algorithm. It starts with a two-second selection time, which decreases to a predefined minimum value with each correct letter choice. The total number of errors presented in the table shows that adjusting the dwell time to the current user skills has a considerable impact on the fluency of text input and the overall level of user satisfaction. Moreover, when gaining more experience, the adaptive time may drop below 1 second, ensuring increasing typing speed, which would be impossible for the statically set selection time.



Fig. 4. The graphical representation of the results – typing speed on the left and error number on the right.

The differences in the results obtained for the particular groups were also verified regarding their statistical significance. At first, the results were subjected to the Shapiro-Wilk test to check the normality of their distributions. Based on the statistic values calculated for the typing speed, the null hypothesis could not be rejected at the assumed significance level, the p-value less than 0.05. It confirmed that the samples came from a population with a normal distribution. The same was observed for the error number in most of the tested groups. However, the alternative hypothesis was adopted in the case of the adaptive selection time for the Side keyboard – the samples did not come from a population with a normal distribution. Then, the ANOVA and Kruskal-Wallis tests were performed depending on the result obtained in the Shapiro-Wilk test. Moreover, for more detailed analysis, the Tukey HSD and Dunn tests were applied for the ANOVA and the Kruskal-Wallis tests, respectively.

The analysis concerning typing speed did not reveal statistically significant differences in the results. However, when the error number was inspected, such significance was presented. The exploration was conducted taking two points of view into account: (1) for each dwell time and all typing methods and then (2) for each keyboard and all dwell times.

The results for the first group of the tests are presented in Table 3. They were evaluated using the ANOVA test for the two- and one-second selection time. For the last option – adaptive one – the Kruskal-Wallis test was performed.

Table 3. F statistics and p-values for ANOVA test, H statistics for Kruskal-Wallis one, for dwell time and the number of errors made.

	Two-second selection	One-second-selection	Adaptive time selection
Statistic F/H	6.6338	5.0689	10.8486
p-value	0.007	0.018	0.004

These outcomes show statistical significance in the results achieved for all selection types. However, the detailed analysis by Tukey HSD and Dunn tests revealed that these differences regarded only the Pie and Tree keyboards for static selection times and (Pie, Tree) and (Pie, Side) for the adaptive time.

Similar dependencies were uncovered when studying the results achieved for all dwell times for each keyboard independently. As shown in Table 4, the differences in the number of errors turned out to be statistically significant.

Table 4. F statistics and p-values for ANOVA test, H statistics for Kruskal-Wallis one, for each keyboard and the number of errors made.

	Pie Keyboard	Tree Keyboard	Side Keyboard
Statistic F/H	3.7071	5.5641	7.7636
p-value	0.045	0.013	0.021

The detailed analysis of these outcomes with the usage of Tukey HSD and Dunn tests (Table 5) confirmed the statistically significant differences for the one-second and the adaptive selection times for all keyboards. It means that the adaptive dwell time allows fewer errors to be made than the one-second choice. The same regards two- and one-second selections for the Tree keyboard. However, also for the Side keyboard, the test result was close to the assumed significance level.

Table 5. P-values for Tukey HSD and Dunn tests for the number of errors made for all keyboards. 2S, 1S, and AD denote the dwell times used.

	p-value		
Selection Time Pairs	Pie Keyboard	Tree Keyboard	Side Keyboard
2S — 1S	0.205	0.043	0.088
2S — AD	0.649	0.889	0.291
1S — AD	0.039	0.016	0.006

6. Summary

The presented studies used the modified approach for determining adaptive dwell time when typing text using an on-screen keyboard. The analysis was conducted with the usage of three typing methods. The proposed solution was explored in terms of typing speed improvement and reducing errors number. The adaptive dwell time was compared to two statically defined parameters: one- and two-second key choices. The research revealed that the one-second dwell time made typing text the fastest. Unfortunately, this was sometimes accompanied by the phenomenon of the Midas touch, which caused the subjects to make unwanted selections. It translated into a large number of mistakes and thus increased users' tiredness. On the other hand, the two-second dwell time, as being too long, unnecessarily lengthened the text input process. According to the obtained outcomes and subjective opinions of the experiment participants, the adaptive selection time proved to be a valuable solution for which the error number was significantly reduced. Such findings concerned all keyboard types.

An important element of text typing systems is the psychological aspect of being rewarded by a fast and seamless text entry. During the experiments, it was noticeable that many errors frustrated the subjects, especially for the Pie keyboard. In the case of the Tree-based one, the users pointed out buttons placed in the corners as more challenging to choose than others. For the Side keyboard, suggestions were made to change the arrangement of the letters to the alphabetical one. However, it must be emphasized that using a given keyboard layout on a daily basis makes users more experienced, alleviating its disadvantages. On the other hand, providing mechanisms for adapting dwell time to current users' skills may encourage them to use the keyboard more frequently, which in turn entails a growing experience.

The overall results are motivating; thus, further improvements are planned. They include an analysis of various dwell time increasing/decreasing steps and considerations regarding the location of the key buttons and letters configuration.

Acknowledgements

This work was supported by Silesian University of Technology, Statutory Research funds of Department of Applied Informatics, Grant Number: 02/100/*BK*_23/0027.

References

[1] Aulikki, H., Howell, I., Stephen, V., 2012. Gaze gestures or dwell-based interaction?, in: Proceedings of the Symposium on Eye Tracking Research and Applications, ETRA '12, pp. 229–232.

[2] Casarini, M., Porta, M., Dondi, P., 2020. A gaze-based web browser with multiple methods for link selection, in: ACM Symposium on Eye Tracking Research and Applications, Association for Computing Machinery, New York, NY, USA. doi:10.1145/3379157.3388929.

[3] Cecotti, H., Meena, Y.K., Prasad, G., 2018. A multimodal virtual keyboard using eye-tracking and hand gesture detection, in: 2018 40th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), pp. 3330–3333. doi:10.1109/EMBC.2018.8512909.

[4] Emiliano, M., Tianyu, W., Campbell, A.T., 2010. Eyephone: Activating mobile phones with your eyes, in: ACM Press, pp. 15–20.

[5] Harezlak, K., Basek, P., Kasprowski, P., 2022. Side keyboard - the new approach for eye-typing, in: Cristani, M., Toro, C., Zanni-Merk, C., Howlett, R.J., Jain, L.C. (Eds.), Knowledge-Based and Intelligent Information & Engineering Systems: Proceedings of the 26th International Conference KES-2022, Verona, Italy and Virtual Event, 7-9 September 2022, Elsevier. pp. 3348–3357. doi:10.1016/j.procs.2022.09.393.

[6] Harezlak, K., Duliban, A., Kasprowski, P., 2021. Eye movement-based methods for human-system interaction. a comparison of different approaches. Procedia Computer Science 192, 3099–3108. doi:https://doi.org/10.1016/j.procs.2021.09.082. knowledge-Based and Intelligent Information Engineering Systems: Proceedings of the 25th International Conference KES2021.

[7] Huckauf, A., Urbina, M.H., 2008. Gazing with peyes: Towards a universal input for various applications, in: Proceedings of the 2008 Symposium on Eye Tracking Research amp; Applications, Association for Computing Machinery, New York, NY, USA. p. 51–54. doi:10.1145/1344471.1344483.

[8] Kasprowski, P., Harezlak, K., Niezabitowski, M., 2016. Eye movement tracking as a new promising modality for human computer interaction, in: 2016 17th International Carpathian Control Conference (ICCC), pp. 314–318.

[9] Krolak, A., Strumillo, P., 2011. Eye-blink detection system for human–computer interaction. Universal Access in the Information Society 11, 1–11.

[10] Majaranta, P., Ahola, U.K., Špakov, O., 2009a. Fast gaze typing with an adjustable dwell time, Association for Computing Machinery, New York, NY, USA. p. 357–360. doi:10.1145/1518701.1518758.

[11] Majaranta, P., Ahola, U.K., Špakov, O., 2009b. Fast gaze typing with an adjustable dwell time, in: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, Association for Computing Machinery, New York, NY, USA. p. 357–360. doi:10.1145/1518701.1518758.

[12] Majaranta, P., Aula, A., Räihä, K.J., 2004. Effects of feedback on eye typing with a short dwell time, in: Proceedings of the 2004 Symposium on Eye Tracking Research amp; Applications, Association for Computing Machinery, New York, NY, USA. p. 139–146. doi:10.1145/968363.968390.

[13] Meena, Y.K., Cecotti, H., Wong-Lin, K., Prasad, G., 2019. Design and evaluation of a time adaptive multimodal virtual keyboard. Journal on Multimodal User Interfaces 13, 343–361.

[14] Morimoto, C.H., Leyva, J.A.T., Diaz-Tula, A., 2018. Context switching eye typing using dynamic expanding targets, in: Proceedings of the Workshop on Communication by Gaze Interaction, Association for Computing Machinery, New York, NY, USA. doi:10.1145/3206343.3206347.

[15] Mott, M.E., Williams, S., Wobbrock, J.O., Morris, M.R., 2017. Improving dwell-based gaze typing with dynamic, cascading dwell times, in: Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems, Association for Computing Machinery, New York, NY, USA. p. 2558–2570. doi:10.1145/3025453.3025517.

[16] Nayyar, A., Dwivedi, U., Ahuja, K., Rajput, N., Nagar, S., Dey, K., 2017. Optidwell: Intelligent adjustment of dwell click time, in: Proceedings of the 22nd International Conference on Intelligent User Interfaces, Association for Computing Machinery, New York, NY, USA. p. 193–204. doi:10.1145/3025171.3025202.

[17] Penkar, A.M., Lutteroth, C., Weber, G., 2012. Designing for the eye: Design parameters for dwell in gaze interaction, in: Proceedings of the 24th Australian Computer-Human Interaction Conference, pp. 479–488.

[18] Pi, J., Shi, B.E., 2017. Probabilistic adjustment of dwell time for eye typing, in: 2017 10th International Conference on Human System Interactions (HSI), pp. 251–257. doi:10.1109/HSI.2017.8005041.

[19] Urbina, M.H., Huckauf, A., 2007. Dwell time free eye typing approaches, in: Proceedings of the 3rd Conference on Communication by Gaze Interaction (COGAIN 2007), pp. 65–70.

[20] Vaitukaitis, V., Bulling, A., 2012. Eye gesture recognition on portable devices. UbiComp'12 - Proceedings of the 2012 ACM Conference on Ubiquitous Computing , 711–714.

[21] Špakov, O., Miniotas, D., 2004. On-line adjustment of dwell time for target selection by gaze, in: Proceedings of the Third Nordic Conference on Human-Computer Interaction, Association for Computing Machinery, New York, NY, USA. p. 203–206. doi:10.1145/1028014.1028045.

[22] Xuebai, Z., Xiaolong, L., Shyan-Ming, Y., Shu-Fan, L., 2017. Eye tracking based control system for natural human-computer interaction. Computational Intelligence and Neuroscience 2017, 1–9.